

QSplitFL: Capability Aware Deep Q-Learning for Optimal Split Point Selection in Split Federated Learning

Nazmus Shakib Shadin¹[0000–0002–0999–924X], Xinyue Zhang¹[0000–0002–4243–083X], Jingyi Wang²[0000–0002–4988–5626], and Miao Pan³[0000–0003–2138–4413]

¹ Department of Computer Science, Kennesaw State University, Marietta, GA, 30060 USA

`nshadin@students.kennesaw.edu`, `xzhang48@kennesaw.edu`

² Department of Computer Science, San Francisco State University, San Francisco, CA, 94132 USA

`jingyiwang@sfsu.edu`

³ Department of Electrical and Computer Engineering, University of Houston, Houston, TX, 77204 USA

`mpan2@uh.edu`

Abstract. Federated Learning (FL) combined with Split Learning (SL) is a privacy preserving paradigm that enables training deep neural networks (DNNs) on resource constrained devices while reducing overall training cost. However, determining the optimal split point, meaning the layer where the model is divided still remains a critical challenge, especially when clients have heterogeneous hardware capabilities. Fixed split points can overload weak devices and increase the communication and server load, which slows convergence and reduces stability. This paper introduces QSplitFL, a novel capability-aware Deep Q-Network (DQN) framework for optimal split point selection in Split learning based Federated Learning (SFL) environments. Unlike existing approaches that rely on high-dimensional model weight representations, QSplitFL employs a lightweight state representation derived directly from client hardware metrics, including CPU utilization, memory, battery level, and network latency. The proposed framework incorporates a decayed loss-drop reward function that prioritizes early convergence, and a committee-based DQN architecture with majority voting to mitigate reward hacking. Extensive experiments on MNIST, Fashion-MNIST, CIFAR-10, and CIFAR-100 datasets using CNN, ResNet50, MobileNetV4, and ConvNeXt architectures demonstrate that our approach achieves better convergence and higher accuracy compared to existing methods, while effectively adapting to heterogeneous device resources. The source code is publicly available at <https://github.com/AIPO-Lab/QSplitFL>.

1 Introduction

The rapid adoption of edge devices along with growing concerns about data privacy and security have driven interest in Federated Learning (FL), which en-

ables collaborative model training without centralizing sensitive raw data [17]. FL is particularly relevant in Internet of Things (IoT), Internet of Medical Things (IoMT), and edge computing environments, where data are naturally distributed across smartphones, IoT sensors, and medical devices deployed in remote and resource limited settings [13,2]. Despite its benefits, FL relies on client side computation and often assumes that participating clients can train complete neural networks locally, which can fail in real-world scenarios with resource constrained client devices [27]. Considering a practical healthcare scenario where rural hospitals and community health centers aim to collaboratively develop and train deep learning models for a particular disease detection using patient data distributed across multiple facilities [13]. While collaboration is essential for achieving robust models that generalize across diverse patient populations, these facilities often operate with limited computational infrastructure, including legacy hardware, constrained power supplies, and unreliable network connectivity [4]. For such environments, the computational cost associated with training deep neural networks (DNNs) locally becomes the dominant bottleneck, which limits the applicability of conventional FL.

To address this limitation, Split Learning (SL) offers a compelling alternative by partitioning the model between clients and the server, which enables resource constrained devices to execute only the initial layers of the model [10]. The resulting intermediate activations, termed smashed data, are then transmitted to the server, which completes the remaining forward and backward passes. SL reduces client side computation while keeping raw data on the device. Building on this idea, Split learning based federated learning (SFL) integrates the computational efficiency of SL with the privacy guarantees and scalability of FL [27].

Even with these advantages of SFL, it introduces a critical optimization challenge, which is determining the optimal split point, that means the specific layer where the neural network is divided between clients and the server [20]. The split point determines how computation and communication are balanced between clients and the server [27]. When the split is shallow, clients execute only a small portion of the network, which reduces client side compute but increases the size of intermediate activations sent to the server and increases server side processing. On the other hand, when the split is deep, more layers run on the client, which reduces activation transmission but can overload resource constrained devices [21]. So in practice, the split point can't be fixed because client capability changes over time and varies across devices, as well as capability can fluctuate across federated rounds due to battery drain, competing applications, network congestion, and hardware heterogeneity within client clusters [6]. In real-world deployments, some devices can support deeper splits, while others cannot [31]. This context motivates adaptive, capability aware split point selection.

Existing split point selection methods mainly use heuristic rules, exhaustive search, or reinforcement learning that relies on high dimensional state representations derived from model weights [24]. These approaches often introduce nontrivial overhead, slow adaptation, and limited interpretability. Methods that build states from model weights typically require dimensionality reduction, often

via Principal Component Analysis (PCA) [30]. This step adds substantial complexity for collecting weights and computing the projection, which is difficult to justify in resource constraint device deployments.

For mitigating these issues, we propose QSplitFL, a capability aware reinforcement learning (RL) based Deep Q Network framework that selects split points dynamically in SFL settings. Here, we have replaced weight based state representations with a lightweight, interpretable state built from client capability metrics. We formulate split selection as a Markov Decision Process [30] in which the state summarizes aggregated cluster capability using normalized metrics for CPU availability, memory utilization, battery level, and network latency, together with a heterogeneity indicator derived from capability distribution across clients. The action selects a split layer from a feasible range, for example, from mid network layers up to the last executable layer, which aligns with real deployments where devices and hospitals differ widely in computing capacity. Resource constrained sites in rural areas may only execute a small portion of the model, while larger hospitals can support deeper computation. This heterogeneity makes a single fixed split point ineffective and motivates a mechanism that adapts split depth to the capability of each participating client cluster. The reward follows a decayed loss drop objective, where the reward function assigns higher credit to early improvements to speed up initial convergence.

QSplitFL uses standard DQN stabilization techniques as described by Wang et al. [30]. This includes experience replay, where a finite-capacity buffer stores previous transitions to decorrelate gradient updates and allow the model to reuse informative experiences. Stability in temporal difference (TD) learning is further maintained through a target network that is periodically synchronized with the main network. Additionally, the framework employs committee-based action selection to mitigate the problem of reward hacking, which ensures more robust and reliable decision making for action selection [32]. The principal contributions of this paper are summarized as follows:

- **Capability-Aware State:** A lightweight and interpretable state representation based on normalized client capability metrics (CPU, memory, battery, network) and cluster heterogeneity.
- **Decayed Loss Drop Reward:** An exponentially decayed loss drop reward that prioritizes early round improvements to accelerate split point discovery.
- **Committee-Based DQN:** A committee-based DQN with multiple CNN models as a single committee member to vote on action selection to improve robustness and reduce reward hacking.
- **First DQN-Based Adaptive Split Point Selection:** To the best of our knowledge, QSplitFL is the first work to apply Deep Q-Learning for optimal split point selection in SFL.
- **Comprehensive Evaluation:** Extensive experiments on MNIST, Fashion MNIST, CIFAR-10, and CIFAR-100 datasets using CNN, ResNet50, MobileNetV4, and ConvNeXt across 5, 10, 100, and 200 clients, demonstrate scalability and effectiveness of our method.

2 Related Work

SFL combines the computation reduction of SL with the collaborative scalability of FL [27,12]. In SFL, clients perform SL with a server, while client side updates are periodically aggregated in a federated manner. This hybrid design enables local participation with resource-constrained devices, while allowing parallel processing of multiple clients on the server side. Recent SFL research has emphasized optimization and resource management. Fan et al. [9] proposed a multi-agent deep reinforcement learning framework for cloud edge device collaborative SFL that jointly optimizes partitioning, resource allocation, and client scheduling. Yu et al. [35] introduced U-shaped SFL for vehicular environments and used deep reinforcement learning to handle dynamic resource allocation and split selection under mobility constraints. ESFL formulates workload and server resource allocation for heterogeneous wireless devices using optimization techniques tailored to system constraints [37]. Privacy and integrity aspects have also been studied. For example, differential privacy mechanisms that perturb smashed data can reduce the success of label inference attacks [33]. IV-FED uses trusted execution environments to support training integrity in healthcare IoT scenarios [15].

Reinforcement learning (RL), and deep reinforcement learning in particular, has been applied to decision making problems in distributed machine learning systems [14]. Deep Q-Networks (DQN) are widely used for discrete action spaces common in scheduling, allocation, and partitioning tasks [30]. In distributed learning, RL has been used for split point selection, client selection, and task offloading. Goal oriented DNN splitting methods use RL to control split decisions under resource constraints and accuracy objectives [5]. Earlier approaches applied Q-learning to split decisions using PCA compressed weight based representations [24]. RL has also been used to select clients in heterogeneous IoT FL settings to balance convergence and resource usage [34], and to optimize task offloading between edge and cloud resources [36]. Improvements to experience replay, such as prioritized experience replay and diversity aware replay, have further improved sample efficiency in DRL training [25].

Research Gap and Our Contribution: Existing RL-based approaches represent the state using high-dimensional model weights with PCA compression [24], which introduces substantial overhead and limits the scalability. They also represent dynamic client capability changes, which makes split point selection decisions slow to adapt. In addition, single agent decision making can be vulnerable to reward hacking, where the policy exploits weaknesses in the reward signal rather than improving true training performance [32]. Finally, reward formulations that treat all rounds equally can miss the importance of early round decisions for shaping later convergence behavior. QSplitFL addresses these gaps through a lightweight capability-aware state, a committee-based DQN with majority voting, and a decayed loss-drop reward for adaptive split point selection in resource-constrained SFL environments.

3 System Model

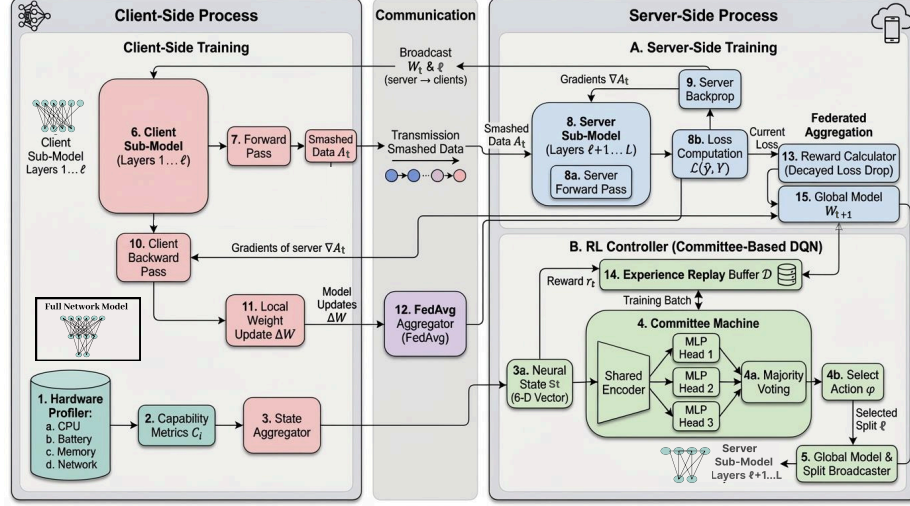


Fig. 1: **QSplitFL Workflow Architecture.** (1) *Client-Side*: Clients has hardware metrics, receive split layer ℓ , and run forward propagation through layers 1 to ℓ , which produces smashed data (A_k). (2) *Server-Side*: The server completes training through layers $\ell + 1$ to L and returns gradients to clients. (3) *Aggregation*: Client updates are aggregated via FedAvg; reward (r_t) is computed on the based of the loss function. (4) *RL Controller*: A committee of MLP models votes on the optimal split layer based on client capabilities.

Figure 1 presents the QSplitFL workflow architecture. On the **Client-Side Process**, each client begins with a Hardware Profiler (Step 1) that measures four real-time metrics: CPU utilization, battery level, memory availability, and network latency. These raw readings are converted into normalized capability metrics C_i (Step 2) and then passed to a state aggregator (Step 3), which constructs a six-dimensional neural state vector s_t (Step 3a) summarizing the cluster’s overall capability and heterogeneity. After receiving the selected split layer ℓ and global model with L layers and weights W_t from the server (Step 5), each client loads its client sub-model with layers 1 to ℓ (Step 6) and runs a forward pass (Step 7) to produce smashed data A_ℓ , which is transmitted to the server.

On the **Server-Side Process**, in Block A, the server receives the smashed data and executes the server forward pass (Step 8a) through the server sub-Model covering layers $\ell + 1$ to L (Step 8). The server calculates $\mathcal{L}(\hat{y}, Y)$ by comparing predictions with true labels (Step 8b). Server backpropagation (Step 9) computes gradients ∇A_ℓ , which are sent back to clients. Each client then performs its client backward pass (Step 10), computes local weight updates ΔW (Step 11),

and sends them to the Aggregator (Step 12) for federated aggregation. The aggregated loss is fed into the reward calculator (Step 13), which computes the decayed loss-drop reward r_t , and the updated global model W_{t+1} (Step 15) is prepared for the next round.

In Block B (**RL Controller**), the server executes the committee-based DQN. The state vector s_t enters the committee stage (Step 4), which consists of a shared encoder feeding into multiple independent MLP Heads. Each head proposes a split action, and majority voting (Step 4a) determines the consensus. The winning action is selected (Step 4b) as the split layer ℓ , which is broadcast to all clients through Step 5. The reward r_t from Step 13 is stored in the experience replay buffer $\mathcal{D}$ (Step 14), from which training batches are sampled to update the committee networks.

Consider a federated network with K edge clients partitioned into C clusters, denoted $\{\mathcal{K}_c\}_{c=1}^C$, where each cluster $\mathcal{K}_c$ contains clients with similar hardware capabilities. The central server maintains the server-side portion of the neural network and coordinates training across all clusters. Within each cluster, a dedicated RL agent (the Q-controller, Block B in Figure 1) observes cluster-level capability states and selects appropriate split points for each training round.

Computation-Communication Trade-off: SFL addresses the computational limitations of resource-constrained devices by allowing each client to execute only the initial layers of the neural network model. The intermediate activations (smashed data) along with the corresponding true labels are transmitted to a central server, which completes the more computation-intensive portions of the training including the deeper layers and loss computation. This approach significantly reduces the workload on devices with limited power, memory, or battery, making participation possible even in resource constrained settings like rural healthcare facilities. Although SFL increases communication costs because activations and gradients must be exchanged between clients and the server; this trade-off is necessary and acceptable [26]. In critical applications like collaborative AI in healthcare, where enabling weak devices to participate matters more than reducing network latency. If a device cannot run the full model locally, the lower communication cost of traditional FL is irrelevant as the device simply cannot participate. QSplitFL optimizes this trade-off by adaptively choosing split points that balance client computation against communication overhead.

4 QSplitFL Framework

4.1 Per-Round Operation

Before formalizing the individual components, we first describe how they interact within a single training round, which gives an end-to-end view of the framework. Figure 2 provides a detailed view of how the QSplitFL framework operates during each training round. It shows the integration of capability-aware state construction (corresponding to Steps 1–3 in Figure 1), RL-based split point selection (Step 4), SFL execution (Steps 5–11), and federated aggregation with reward computation (Steps 12–15). The per-round workflow proceeds as follows:

1. **State Construction:** At the beginning of each round, the Hardware Profiler (Step 1) collects capability metrics (CPU, memory, battery, network) from all clients, which are normalized into Capability Metrics C_i (Step 2) and aggregated by the State Aggregator (Step 3) into the six-dimensional neural state s_t .
2. **Action Selection:** The RL Controller’s Committee Machine evaluates the current state through a Shared Encoder and multiple MLP Heads. Majority Voting (Step 4a) determines the consensus split layer, and the selected action is output (Step 4b).
3. **SFL Execution:** The Global Model and Split Broadcaster (Step 5) sends W_t and ℓ to all clients. Each client loads its Sub-Model (Step 6) and runs the Forward Pass (Step 7) to produce smashed data A_ℓ . The server executes the Server Forward Pass (Step 8a) through layers $\ell + 1$ to L , computes the loss (Step 8b), and performs Server Backpropagation (Step 9). Gradients ∇A_ℓ are returned to clients for the Client Backward Pass (Step 10) and Local Weight Update (Step 11).
4. **Federated Aggregation:** The FedAvg Aggregator (Step 12) combines all client updates. The Reward Calculator (Step 13) computes the decayed loss-drop reward r_t , which is stored in the Experience Replay Buffer $\mathcal{D}$ (Step 14). The updated Global Model W_{t+1} (Step 15) is prepared for the next round.

The QSplitFL Architecture Execution in Each Round

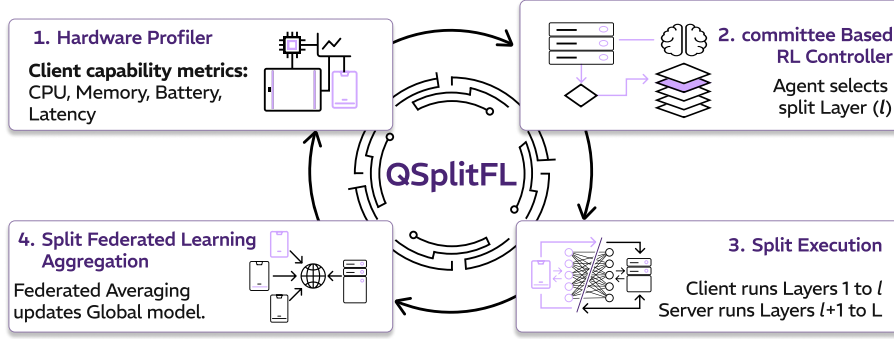


Fig. 2: High-Level Overview of QSplitFL Framework Operation in Each Training Round. The diagram illustrates the complete workflow: (1) capability metric collection and state construction, (2) committee-based split point selection, (3) SFL execution with smashed data transmission, and (4) federated aggregation with reward computation.

With this per-round view in place, we now formalize each component, beginning with the Markov Decision Process formulation that underpins split point selection.

4.2 Markov Decision Process Formulation

We formulate the split point selection problem as a Markov Decision Process (MDP) defined by the tuple $(\mathcal{S}, \mathcal{A}, P, R, \gamma)$, where $\mathcal{S}$ denotes the state space, $\mathcal{A}$ the action space, P the transition, R the reward function, and γ the discount factor. The MDP connects the key blocks shown in Figure 1: the state is constructed from client capability metrics (Steps 1–3), the action determines the split layer via the RL Controller (Step 4), the transition executes one SFL round (Steps 5–12), and the reward is derived from the aggregated loss (Step 13).

State Space: The state at round t for cluster c encodes the aggregated hardware capabilities and heterogeneity of participating clients. As shown in Steps 1–3 of Figure 1, each client’s Hardware Profiler (Step 1) reports raw metrics, which are normalized into Capability Metrics C_i (Step 2) and aggregated by the State Aggregator (Step 3) into a compact neural state vector s_t . Unlike prior approaches that employ high-dimensional model weight representations requiring PCA compression, we define a compact, interpretable state vector based on four normalized capability metrics. For each client k at time t , we compute: CPU availability as $C_{\text{CPU}}^{(k)}(t) = \text{CPU}_{\text{avail}}^{(k)}(t)/\text{CPU}_{\text{max}}$, memory availability as $C_{\text{Mem}}^{(k)}(t) = \text{Mem}_{\text{avail}}^{(k)}(t)/\text{Mem}_{\text{max}}$, battery level as $C_{\text{Bat}}^{(k)}(t) = \text{Bat}_{\text{level}}^{(k)}(t)/\text{Bat}_{\text{max}}$, and network quality as $C_{\text{Net}}^{(k)}(t) = 1 - \text{Latency}^{(k)}(t)/\text{Latency}_{\text{max}}$. All metrics are normalized to the interval $[0, 1]$ using min-max normalization across the federated network; see Appendix A.2 for detailed descriptions. The overall capability score for client k combines these metrics as $C_{\text{Overall}}^{(k)}(t) = \sum_{i=1}^4 w_i \cdot C_i^{(k)}(t)$, where $w_1, w_2, w_3, w_4 \geq 0$ and $\sum_{i=1}^4 w_i = 1$ are importance weights which are tunable according to the deployment requirements. The cluster-level state vector aggregates individual client metrics into a six-dimensional representation: $s_t^{(c)} = [\bar{C}_{\text{CPU}}^{(c)}(t), \bar{C}_{\text{Memory}}^{(c)}(t), \bar{C}_{\text{Battery}}^{(c)}(t), \bar{C}_{\text{Network}}^{(c)}(t), \bar{C}_{\text{Overall}}^{(c)}(t), \sigma_c(t)]$. Here, each mean capability is computed as $\bar{C}_i^{(c)}(t) = \frac{1}{|\mathcal{K}_c|} \sum_{k \in \mathcal{K}_c} C_i^{(k)}(t)$, which represents the average of metric i across all clients in cluster c . The heterogeneity indicator $\sigma_c(t) = \sqrt{\frac{1}{|\mathcal{K}_c|} \sum_{k \in \mathcal{K}_c} (C_{\text{Overall}}^{(k)}(t) - \bar{C}_{\text{Overall}}^{(c)}(t))^2}$ captures the standard deviation of overall capability scores within the cluster. This six-dimensional state representation provides direct interpretability: high $\bar{C}_{\text{Overall}}^{(c)}(t)$ indicates the cluster can accommodate deeper split points, while high $\sigma_c(t)$ shows heterogeneous capabilities which require adaptive split point selection to avoid overloads from weaker client devices.

Action Space: The action space consists of feasible split layer indices $\mathcal{A} = \{\ell \in \mathbb{N} : \ell_{\min} \leq \ell \leq \ell_{\max}\}$, where $\ell_{\min} = \lceil L/2 \rceil$ represents the minimum split point set at approximately half the total network depth L , and $\ell_{\max} = L - 1$ is the maximum split point. The RL Controller’s Committee Machine (Step 4 in Figure 1) selects one split layer from this range through majority voting (Step 4a) and outputs the selected action (Step 4b). The lower bound $\ell_{\min} = \lceil L/2 \rceil$ ensures

that clients execute at least half of the network layers, which serves two purposes: (1) it prevents clients from sending raw or nearly-raw data to the server, which would increase privacy risks and communication overhead; and (2) it ensures client devices perform meaningful local feature extraction before transmitting smashed data. For example, in ResNet50 with $L = 50$ layers, this constraint sets the split point at least in the half layer ($\ell_{\min} = 25$). The upper bound $L - 1$ excludes the output layer, which must reside on the server for loss computation and gradient initialization. Each action $a_t = \ell$ specifies that clients compute layers 1 through ℓ while the server computes layers $\ell + 1$ through L .

Transition: After the RL Controller selects a split layer ℓ at round t (Step 4 in Figure 1), the environment executes one complete SFL training round following Steps 5 through 12 as shown in Figure 1, using Algorithm 2 from Appendix A.7:

$$W_t = \text{TrainRoundSFL}(c, s_t^{(c)}, W_{t-1}; Q_\theta, \epsilon), \quad (1)$$

where the server broadcasts parameters and split configuration (Step 5), clients perform forward passes and transmit smashed data (Steps 6–7), the server completes forward and backward passes (Steps 8–9), clients perform backpropagation and weight updates (Steps 10–11), and updates are aggregated via FedAvg (Step 12). The next state is then computed:

$$s_{t+1}^{(c)} = \Phi \left(\{C_i^{(k)}(t+1)\}_{k \in \mathcal{K}_c} \right), \quad (2)$$

where $\Phi(\cdot)$ applies the aggregation to updated client capability metrics.

4.3 Reward Function Design

The reward function guides the RL agent toward optimized split point selections that minimize cluster loss while considering capability constraints. As indicated in Step 13 of Figure 1, the Reward Calculator computes the reward (r_t) based on the decayed loss-drop during the aggregation phase. We employ a decayed loss-drop reward that prioritizes early-round improvements.

Cluster Loss Computation: At each round t , we calculate the overall validation loss for a cluster by averaging the losses from all clients, where we assign more weight to clients with more data. The cluster loss is computed as $L_t = \sum_{k=1}^K \omega_k \cdot L_t^{(k)}$, where $\omega_k = \frac{n_k}{\sum_{j=1}^K n_j}$ represents the weight of client k based on its number of training samples n_k .

Decay Factor: Since the model improves most in early rounds, we use an exponential decay factor to give more credit to early round gains and less to the later ones for optimal split point selection. So, the exponential decay factor can be defined as, $\rho_t = e^{-\lambda(t-1)}$, where $\lambda > 0$. Here, at round $t = 1$, we have

$\rho_1 = e^0 = 1$, ensuring the first round receives full credit. As training progresses, ρ_t decays exponentially toward zero, causing later rounds to receive diminishing credit for equivalent loss improvements. The decay rate λ controls the speed of this transition: larger values of λ result in sharper decay, while smaller values produce more gradual decay. In practice, we set λ such that $\rho_T \ll 1$ by the final round, which ensures the RL agent prioritizes early stage convergence.

Loss-Drop Reward: The reward at round t measures the improvement in cluster loss, weighted by the temporal decay: $r_t = -(L_t - L_{t-1}) \cdot \rho_t$. If the selected split point leads to loss reduction ($L_t < L_{t-1}$), the reward is positive; and if the loss increases it yields negative reward. The decay factor ρ_t ensures that equivalent improvements in later rounds receive progressively smaller rewards, encouraging the agent to prioritize early convergence. Detailed reward behavior examples are provided in Table 4 in Appendix A.6.

4.4 Deep Q-Network Architecture

The RL Controller shown in Block B of Figure 1 is implemented using a Deep Q-Network (DQN) to approximate the action value function $Q(s, a)$. The Q-network is an MLP that maps the six-dimensional capability state (constructed through Steps 1–3) to Q-values for each possible split action. To improve learning stability, we maintain an experience replay buffer $\mathcal{D}$ that stores transition tuples $e_t = (s_t, a_t, r_t, s_{t+1}, d_t)$, where $d_t \in \{0, 1\}$ indicates round termination. The buffer operates under a First-In-First-Out (FIFO) policy with fixed capacity. During training, mini-batches of B transitions are sampled uniformly from $\mathcal{D}$, which breaks the correlation between consecutive samples and helps the model learn more effectively. We also use a target network $Q_{\bar{\theta}}$ with parameters $\bar{\theta}$ that are periodically synchronized with the current network. For each sampled transition $(s_i, a_i, r_i, s'_i, d_i)$, the TD target is computed as $y_i = r_i + \gamma(1 - d_i) \max_{a'} Q_{\bar{\theta}}(s'_i, a')$. The DQN parameters are updated by minimizing the mean-squared TD error in B mini-batches is:

$$\mathcal{L}(\theta) = \frac{1}{B} \sum_{i=1}^B (Q_{\theta}(s_i, a_i) - y_i)^2. \quad (3)$$

For action selection, we use the standard ϵ -greedy exploration strategy with exponential decay [11].

Committee-Based Solution for Mitigating Reward Hacking: A single agent can sometimes learn to cheat by finding shortcuts that increase its reward without actually improving the model’s real performance; this phenomenon is known as reward hacking [32]. As shown in Step 4 of Figure 1, the RL Controller’s Committee Machine employs M MLP Heads with a Shared Encoder (with M being an odd number) that vote together via Majority Voting (Step 4a) to decide the best split action (Step 4b). Each committee member shares a common

Algorithm 1 SFL with RL-based DQN Training

Input: M (odd), $L, \mathcal{A}, \eta, \gamma, \epsilon_0, \epsilon_{\min}, \kappa, \mathcal{C}, N_{\max}, B, \{\mathcal{K}_c\}, T, E$
Output: Trained committee $\{Q_\theta^{(m)}\}_{m=1}^M$; policy π

 1: **Model form (shared encoder, per-member head):**

$$f_s(\cdot; \phi_s) \text{ (shared)}, \quad g^{(m)}(\cdot; \psi^{(m)}) \text{ (head)}$$

$$Q_\theta^{(m)}(s, a) = [g^{(m)}(f_s(s; \phi_s); \psi^{(m)})]_a; \theta_m = f_s + g^{(m)}$$

```

2: Initialize shared encoder  $f_s(\cdot; \phi_s)$ , heads  $\{g^{(m)}\}_{m=1}^M$ ; set  $Q_{\bar{\theta}}^{(m)} \leftarrow Q_\theta^{(m)}$  for all  $m$ 
3: Initialize replay buffer  $\mathcal{D} \leftarrow \emptyset$ 
4: for episode  $e = 1$  to  $E$  do
5:   for cluster  $c = 1$  to  $C$  do
6:     Set  $\epsilon \leftarrow \epsilon_0$ ;  $L_0 \leftarrow \infty$ 
7:     for round  $t = 1$  to  $T$  do
8:       Compute capability metrics  $C_{\text{CPU}}^{(k)}(t), C_{\text{Memory}}^{(k)}(t), C_{\text{Battery}}^{(k)}(t), C_{\text{Network}}^{(k)}(t)$ 
       for all  $k \in \mathcal{K}_c$ 
9:       Construct state  $s_t^{(c)} = [\bar{C}_{\text{CPU}}^{(c)}, \bar{C}_{\text{Memory}}^{(c)}, \bar{C}_{\text{Battery}}^{(c)}, \bar{C}_{\text{Network}}^{(c)}, \bar{C}_{\text{Overall}}^{(c)}, \sigma_c(t)]$ 
10:      Select action  $a_t$ : with prob.  $\epsilon$  random from  $\mathcal{A}$ , else  $a_t \leftarrow$ 
      CommitteeVote( $s_t^{(c)}, \{Q_\theta^{(m)}\}$ ) (Alg. 3)
11:      Execute SFL round:  $(W_t, L_t) \leftarrow \text{TrainRoundSFL}(c, s_t^{(c)}, W_{t-1}, \ell = a_t)$ 
      (Alg. 2)
12:      Compute reward  $r_t = -(L_t - L_{t-1}) \cdot \rho_t$  where  $\rho_t = e^{-\lambda(t-1)}$ 
13:      Obtain next state  $s_{t+1}^{(c)}$  by recomputing capability metrics
14:      Set terminal flag:  $d_t = \mathbf{1}$  [if  $t = T$ ]
15:      Store  $(s_t^{(c)}, a_t, r_t, s_{t+1}^{(c)}, d_t)$  in  $\mathcal{D}$ ; if  $|\mathcal{D}| > N_{\max}$ , pop oldest
16:      Sample mini-batch  $\{(s_i, a_i, r_i, s'_i, d_i)\}_{i=1}^B$  from  $\mathcal{D}$ 
17:      for  $m = 1$  to  $M$  do
18:        Compute TD target:  $y_i^{(m)} = r_i + \gamma(1 - d_i) \max_{a'} Q_{\bar{\theta}}^{(m)}(s'_i, a')$ 
19:        Update:  $\theta^{(m)} \leftarrow \theta^{(m)} - \eta \nabla_{\theta^{(m)}} \frac{1}{B} \sum_i (Q_\theta^{(m)}(s_i, a_i) - y_i^{(m)})^2$ 
20:        if  $t \bmod \mathcal{C} = 0$  then
21:           $Q_{\bar{\theta}}^{(m)} \leftarrow Q_\theta^{(m)}$ 
22:        end if
23:      end for
24:      Decay:  $\epsilon \leftarrow \epsilon_{\min} + (\epsilon_0 - \epsilon_{\min})e^{-\kappa t}$ ;  $L_{t-1} \leftarrow L_t$ 
25:    end for
26:  end for
27: end for
28: return  $\{Q_\theta^{(m)}\}_{m=1}^M, \pi(s) = \text{CommitteeVote}(s, \{Q_\theta^{(m)}\})$ 

```

shallow encoder $f_s(\cdot; \phi_s)$ for stable low-level state representation, but maintains an independent deep head $g^{(m)}(\cdot; \psi^{(m)})$ for diverse Q-value estimation:

$$Q_\theta^{(m)}(s, a) = [g^{(m)}(f_s(s; \phi_s); \psi^{(m)})]_a. \quad (4)$$

By sharing the early layers but keeping separate deeper layers, the committee members learn different perspectives on which actions are best, without adding

too much computational cost. During action selection, each committee member proposes its preferred split point by selecting the action with the highest Q-value. We then count the votes for each action and select the action with the most votes as the final decision. In case of ties, we break by selecting the action with the highest mean Q-value across committee members. This majority voting mechanism ensures that no single model can exploit loopholes in the reward function. It is worth clarifying the relationship between the committee structure and the broader RL formulation. QSplitFL is designed as a single-agent RL system. A single centralized RL Controller on the server observes one global state $s_t^{(c)}$ aggregating all client capability metrics and outputs one global action, the split layer ℓ , applied uniformly in that round. The M MLP heads are not independent agents. Rather, they form an ensemble of Q-value estimators within the single agent, each sharing the same encoder but maintaining separate deeper layers [32]. A true multi-agent formulation would require each client to act as an independent decision maker, introducing non-stationary dynamics and coordination overhead. Since QSplitFL needs one globally consistent split point per cluster at each round, the single-agent MDP formulation is both sufficient and appropriate. Beyond architectural clarity, this ensemble design serves a critical functional purpose which is mitigating reward hacking. To verify this, we conducted a controlled comparison between a single-head DQN ($M=1$) and the committee DQN ($M=3$). The single-head agent converged to a fixed split point early in training, accumulating high reward through early loss-drops while accuracy plateaued. The committee DQN, in contrast, continued adapting its split selection across rounds and achieved significantly higher final accuracy, which validates majority voting as an effective defense against reward hacking.

4.5 Training Procedure

Algorithm 1 presents the main controller that orchestrates this complete training loop. It manages the experience replay buffer $\mathcal{D}$ and coordinates all committee members. At each round t , it computes capability metrics, constructs the six-dimensional state $s_t^{(c)}$, and determines whether to explore (random action with probability ϵ) or exploit (committee voting with probability $1 - \epsilon$). Two supporting algorithms are provided in Appendix A.7: Algorithm 2 (SFL Training Round) executes the FL round corresponding to Steps 5–12 in Figure 1, and Algorithm 3 (Committee Majority Voting) implements Step 4.

4.6 Computational Complexity Analysis

A key advantage of the capability-based state representation is its computational efficiency compared to weight-based approaches. Traditional methods using PCA-compressed model weights require high complexity but our capability-based states require only $\mathcal{O}(|\mathcal{K}|)$, which corresponds to simple aggregation operations (mean, standard deviation) over client capability metrics. This becomes especially significant in large scale settings when the network has many layers (d) and there are many participating clients ($|\mathcal{K}|$).

5 Experiments and Results

5.1 Experimental Setup

Datasets and Architectures: We evaluate QSplitFL on MNIST, Fashion-MNIST, CIFAR-10, and CIFAR-100, covering a range of complexity from simple grayscale to challenging RGB classification tasks. To assess performance across varying computational settings, we have paired these datasets with four DNN architectures: a 10-layer CNN (split points 5–9), ResNet50 (50 layers, split points 25–49), MobileNetV4 (53 layers, split points 27–52), and ConvNeXt (59 layers, split points 30–58).

Heterogeneity and Reproducibility: To emulate heterogeneous, resource-constrained clients, each client samples its four capability metrics from device-tier distributions (strong, medium, and weak) that fluctuate across rounds, and clients are grouped into capability clusters served by a per-cluster controller. Data are partitioned with a Dirichlet distribution ($\alpha = 0.5$) for non-IID conditions, and for fairness all baselines share the same backbone, data partition, optimizer, and number of rounds, with split-based baselines using the same client and server partition where applicable. The full device-tier ranges, clustering procedure, hyperparameter settings, and baseline tuning protocol are detailed in Appendix A.8, and Appendix A.11 discusses the rationale for the split depth bound, its privacy implications, system-level cost, feature selection, and limitations.

5.2 Accuracy Convergence Analysis (100 Rounds)

Figure 3 shows how accuracy improves over 100 training rounds for all datasets and architectures (except CNN). Deeper models consistently outperform shallow CNNs. ConvNeXt achieves the best results: 99.6% on MNIST, 94.1% on Fashion-MNIST, 86.5% on CIFAR-10, and 68.3% on CIFAR-100. Across all datasets, we observe that accuracy increases sharply in the first 20–30 rounds and then gradually stabilizes. This pattern holds regardless of the number of clients (5, 10, 50, 100, or 200), showing that QSplitFL scales well to larger federated settings. The performance gap between architectures becomes more noticeable on harder datasets like CIFAR-100, where ConvNeXt performs better than ResNet50 and MobileNetV4. CNN results and additional comparisons are provided in Appendix A.9.

5.3 Split Point Selection Analysis

Figure 4 shows how the RL agent adaptively selects split points for deep architectures across all datasets over 100 rounds. The agent learns to choose higher split points (layers 30–59) for deeper networks like ResNet50, MobileNetV4, and ConvNeXt. This means more layers run on the client side, which is possible because these architectures have more parameters to distribute. The split point selection remains relatively stable across different client counts. This suggests

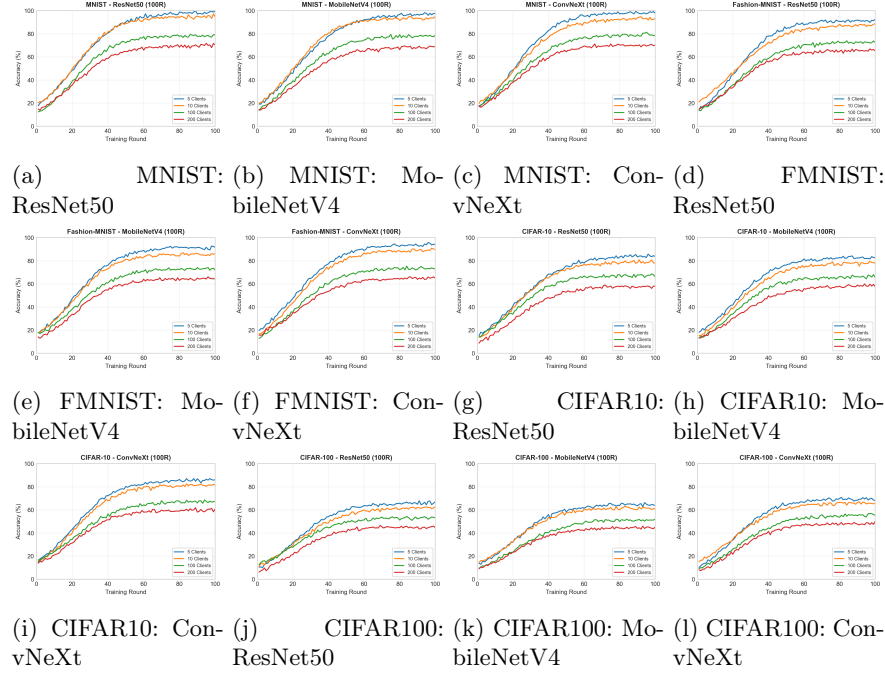


Fig. 3: **Accuracy Convergence Analysis (100 Rounds).** Comparison of ResNet50, MobileNetV4, and ConvNeXt across MNIST, Fashion-MNIST, CIFAR-10, and CIFAR-100 with varying client counts. CNN results are detailed in Appendix A.9.

that the agent focuses primarily on the model architecture rather than the number of participating clients when making split decisions. The consistency of these choices across datasets also indicates that the learned policy generalizes well to different data distributions.

5.4 Comparison with Baseline Methods

We have compared the QSplitFL framework against a few state-of-the-art techniques, such as: Centralized Learning (theoretical upper bound), FedAvg [19], FedProx [17], q-FedAvg [18], SplitFed [27], ClusterSFL [3], HeteroSFL [7], SHerL-FL [28], and FLUID/SFL-V2 [8]. Table 1 presents the comparison of QSplitFL against baseline methods using 5 clients and 100 training rounds. A detailed ablation study isolating the contribution of each design component is provided in Appendix A.10.

5.5 Summary of Findings

Across all datasets and training settings, we observe a consistent performance ranking: ConvNeXt > ResNet50 > MobileNetV4 > CNN. This ordering holds

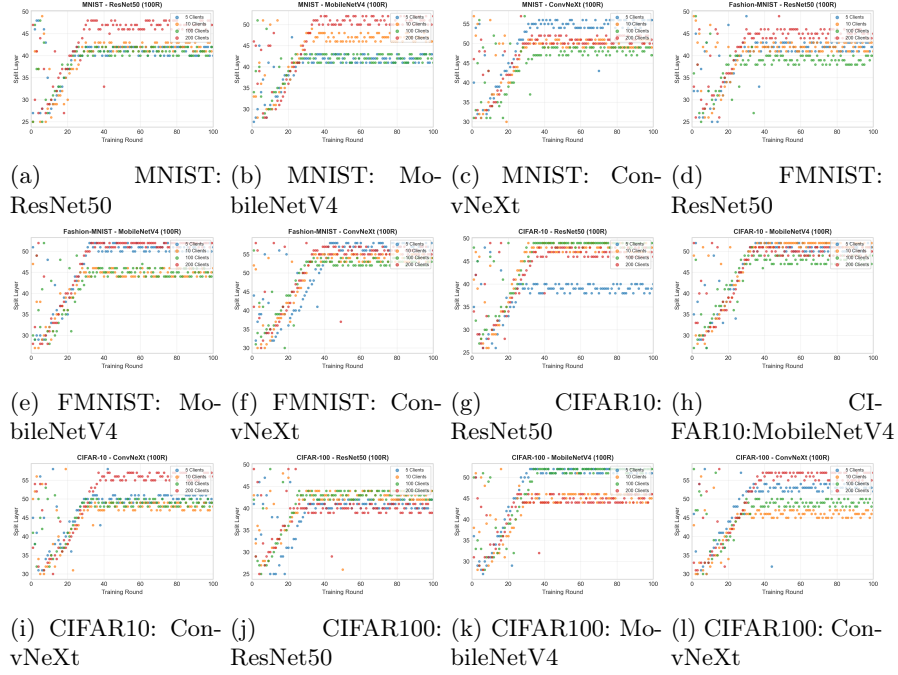


Fig. 4: Dynamic Split Point Selection (100 Rounds). Adaptive split point selection for ResNet50, MobileNetV4, and ConvNeXt. The agent consistently selects optimal split layers (30–59 depending on architecture) to balance computation and communication.

regardless of the model architectures, number of clients, and training rounds, which ensures that deeper and more modern architectures benefit the most from QSplitFL’s adaptive split point selection. Extended training with 100 rounds improves accuracy by 15–28% over 10 rounds, with the largest gains on harder datasets like CIFAR-100. Furthermore, all configurations scale stably from 5 to 200 clients, which demonstrates QSplitFL’s effectiveness in both small and large federated settings. In terms of split behavior, shallow CNNs use early layers (5–9), while deeper networks use mid to late layers (30–59), which enables efficient resource utilization across heterogeneous devices. Beyond accuracy, this adaptive split selection also improves communication efficiency, where deeper splits reduce the size of transmitted activations, while the capability-aware state naturally penalizes poor connectivity clients through $C_{\text{Net}}^{(k)}(t) = 1 - \text{Latency}^{(k)}(t)/\text{Latency}_{\text{max}}$, pushing the agent to favor deeper splits that transmit less data. As a result, QSplitFL implicitly optimizes communication overhead without an explicit bandwidth term in the reward, making QSplitFL well suited for bandwidth-constrained deployments such as rural healthcare facilities and IoT edge environments.

Table 1: Baseline Accuracy Comparison (QSplitFL with 5 Clients, 100 Rounds)

Method	MNIST(%)	FMNIST(%)	CIFAR-10(%)	CIFAR-100(%)	Devices
Centralized	99.8	94.5	93.8	76.5	Server
FedAvg [19]	97.8	89.2	79.5	58.4	High-end
FedProx [17]	98.1	90.1	81.2	60.1	High-end
q-FedAvg [18]	97.5	88.5	78.9	57.8	High-end
SplitFed [27]	99.2	81.0	63.8	–	All
ClusterSFL [3]	–	89.2	78.5	–	All (IoT)
HeteroSFL [7]	–	–	69.45	54.54	Heterogeneous
SHeRL-FL [28]	–	–	65.44	55.23	All
FLUID/SFL-V2 [8]	–	–	69.45	52.38	All
QSplitFL	99.47	92.62	83.73	67.26	All

6 Conclusion

This paper presented QSplitFL, a capability-aware Deep Q-Network framework for dynamic split point selection in SFL. The proposed approach addresses the fundamental challenge of enabling FL on resource-constrained edge devices by intelligently partitioning neural network models based on client hardware capabilities. Our framework introduces three key innovations: (1) a lightweight state representation that reduces computational complexity to $\mathcal{O}(|\mathcal{K}|)$ by directly encoding client metrics instead of model weights, (2) a committee-based RL architecture with majority voting that mitigates reward hacking and ensures adaptive split point decisions, and (3) a decayed loss-drop reward function that prioritizes early-round convergence. Experimental evaluation across four benchmark datasets and four neural network architectures demonstrates that QSplitFL achieves accuracy of 99.47% on MNIST, 93.99% on Fashion-MNIST, 86.16% on CIFAR-10, and 68.27% on CIFAR-100, while scaling gracefully from 5 to 200 clients. Finally, QSplitFL outperforms traditional FL methods while enabling participation of resource-constrained devices.

Acknowledgments. The work is partially supported by the U.S. National Science Foundation under grants NSF-2348417 and NSF-2431597.

References

1. Abadi, M., Chu, A., Goodfellow, I., McMahan, H.B., Mironov, I., Talwar, K., Zhang, L.: Deep learning with differential privacy. In: Proceedings of the 2016 ACM SIGSAC conference on computer and communications security. pp. 308–318 (2016)
2. Abreha, H.G., Hayajneh, M., Serhani, M.A.: Federated learning in edge computing: a systematic survey. *Sensors* **22**(2), 450 (2022)
3. Arafah, M., Wazzeh, M., Sami, H., Ould-Slimane, H., Talhi, C., Mourad, A., Otrok, H.: Efficient privacy-preserving ml for iot: Cluster-based split federated learning scheme for non-iid data. *Journal of Network and Computer Applications* **236**, 104105 (2025)

4. Beutel, D.J., Topal, T., Mathur, A., Qiu, X., Fernandez-Marques, J., Gao, Y., Sani, L., Li, K.H., Parcollet, T., de Gusmão, P.P.B., et al.: Flower: A friendly federated learning research framework. arXiv preprint arXiv:2007.14390 (2020)
5. Binucci, F., Merluzzi, M., Banelli, P., Strinati, E.C., Di Lorenzo, P.: Enabling edge artificial intelligence via goal-oriented deep neural network splitting. In: 2024 19th International Symposium on Wireless Communication Systems (ISWCS). pp. 1–6. IEEE (2024)
6. Chen, H., Chen, X., Peng, L., Bai, Y.: Personalized fair split learning for resource-constrained internet of things. *Sensors* **24**(1), 88 (2023)
7. Chen, X., Li, J., Fan, D., Chakrabarti, C.: Heterosfl: Split federated learning with heterogeneous clients and non-iid data. *IEEE Internet of Things Journal* **12**(15), 30460–30474 (2025). <https://doi.org/10.1109/JIOT.2025.3572393>
8. Dachille, J., Huang, C., Liu, X.: The impact of cut layer selection in split federated learning. arXiv preprint arXiv:2412.15536 (2024)
9. Fan, W., Chen, P., Chun, X., Liu, Y.: Madrl-based model partitioning, aggregation control, and resource allocation for cloud-edge-device collaborative split federated learning. *IEEE Transactions on Mobile Computing* (2025)
10. Gupta, O., Raskar, R.: Distributed learning of deep neural network over multiple agents. *Journal of Network and Computer Applications* **116**, 1–8 (2018)
11. Hariharan, N., Paavai, A.G.: A brief study of deep reinforcement learning with epsilon-greedy exploration. *International Journal of Computing and Digital Systems* **11**(1), 541–552 (2022)
12. Hukkeri, G.S., Goudar, R., Dhananjaya, G., Rathod, V.N.: A comprehensive survey on split-fed learning: methods, innovations, and future directions. *IEEE Access* **13**, 46312 (2025)
13. Khan, L.U., Saad, W., Han, Z., Hossain, E., Hong, C.S.: Federated learning for internet of things: Recent advances, taxonomy, and open challenges. *IEEE Communications Surveys & Tutorials* **23**(3), 1759–1799 (2021)
14. Lee, Y.L., Qin, D.: A survey on applications of deep reinforcement learning in resource management for 5g heterogeneous networks. In: 2019 Asia-Pacific signal and information processing association annual summit and conference (APSIPA ASC). pp. 1856–1862. IEEE (2019)
15. Li, J., Yu, S.: Integrity verifiable privacy-preserving federated learning for healthcare-iot. In: 2024 IEEE International Conference on E-health Networking, Application & Services (HealthCom). pp. 1–6. IEEE (2024)
16. Li, O., Sun, J., Yang, X., Gao, W., Zhang, H., Xie, J., Smith, V., Wang, C.: Label leakage and protection in two-party split learning. arXiv preprint arXiv:2102.08504 (2021)
17. Li, T., Sahu, A.K., Zaheer, M., Sanjabi, M., Talwalkar, A., Smith, V.: Federated optimization in heterogeneous networks. *Proceedings of Machine learning and systems* **2**, 429–450 (2020)
18. Li, T., Sanjabi, M., Beirami, A., Smith, V.: Fair resource allocation in federated learning. arXiv preprint arXiv:1905.10497 (2019)
19. Li, X., Huang, K., Yang, W., Wang, S., Zhang, Z.: On the convergence of fedavg on non-iid data. arXiv preprint arXiv:1907.02189 (2019)
20. Liang, Y., Chen, Q., Zhu, G., Awan, M.K., Jiang, H.: Communication-and-computation efficient split federated learning: Gradient aggregation and resource management. arXiv preprint arXiv:2501.01078 (2025)
21. Liu, X., Deng, Y., Mahmoodi, T.: Wireless distributed learning: A new hybrid split and federated learning approach. *IEEE Transactions on Wireless Communications* **22**(4), 2650–2665 (2022)

22. Nixon, J., Lakshminarayanan, B., Tran, D.: Why are bootstrapped deep ensembles not better? In: "I Can't Believe It's Not Better!" NeurIPS 2020 workshop (2020)
23. Osband, I., Blundell, C., Pritzel, A., Van Roy, B.: Deep exploration via bootstrapped dqn. *Advances in neural information processing systems* **29** (2016)
24. Samikwa, E., Di Maio, A., Braun, T.: Ares: Adaptive resource-aware split learning for internet of things. *Computer Networks* **218**, 109380 (2022)
25. Schaul, T., Quan, J., Antonoglou, I., Silver, D.: Prioritized experience replay. *arXiv preprint arXiv:1511.05952* (2015)
26. Singh, A., Vepakomma, P., Gupta, O., Raskar, R.: Detailed comparison of communication efficiency of split learning and federated learning. *arXiv preprint arXiv:1909.09145* (2019)
27. Thapa, C., Arachchige, P.C.M., Camtepe, S., Sun, L.: Splitfed: When federated learning meets split learning. In: *Proceedings of the AAAI conference on artificial intelligence*. vol. 36, pp. 8485–8493 (2022)
28. Tran, D.T., Ha, N.B., Nguyen, V.D., Wong, K.S.: Sherl-fl: When representation learning meets split learning in hierarchical federated learning. *arXiv preprint arXiv:2508.08339* (2025)
29. Vepakomma, P., Gupta, O., Swedish, T., Raskar, R.: Split learning for health: Distributed deep learning without sharing raw patient data. *arXiv preprint arXiv:1812.00564* (2018)
30. Wang, H., Kaplan, Z., Niu, D., Li, B.: Optimizing federated learning on non-iid data with reinforcement learning. In: *IEEE INFOCOM 2020-IEEE conference on computer communications*. pp. 1698–1707. IEEE (2020)
31. Wu, D., Ullah, R., Harvey, P., Kilpatrick, P., Spence, I., Varghese, B.: Fedadapt: Adaptive offloading for iot devices in federated learning. *IEEE Internet of Things Journal* **9**(21), 20889–20901 (2022)
32. Wu, F., Liu, X., Wang, H., Wang, X., Su, L., Gao, J.: Towards federated rlhf with aggregated client preference for llms. *arXiv preprint arXiv:2407.03038* (2024)
33. Wu, M., Cheng, G., Li, P., Yu, R., Wu, Y., Pan, M., Lu, R.: Split learning with differential privacy for integrated terrestrial and non-terrestrial networks. *IEEE Wireless Communications* **31**(3), 177–184 (2023)
34. Yan, S., Zhang, P., Huang, S., Wang, J., Sun, H., Zhang, Y., Tolba, A.: Node selection algorithm for federated learning based on deep reinforcement learning for edge computing in iot. *Electronics* **12**(11), 2478 (2023)
35. Yu, L., Chang, Z., Jia, Y., Min, G.: Model partition and resource allocation for split learning in vehicular edge networks. *IEEE Transactions on Intelligent Transportation Systems* (2025)
36. Yuan, X., Zhang, Z., Feng, C., Cui, Y., Garg, S., Kaddoum, G., Yu, K.: A dqn-based frame aggregation and task offloading approach for edge-enabled iomt. *IEEE Transactions on Network Science and Engineering* **10**(3), 1339–1351 (2022)
37. Zhu, G., Deng, Y., Chen, X., Zhang, H., Fang, Y., Wong, T.F.: Esfl: Efficient split federated learning over resource-constrained heterogeneous wireless devices. *IEEE Internet of Things Journal* **11**(16), 27153–27166 (2024)